

THE UNIVERSITY OF BAMENDA

**NATIONAL
HIGHER
POLYTECHNIC
INSTITUTE**



**CENTRE FOR
CYBERSECURITY
AND
MATHEMATICAL
CRYPTOLOGY**

Fully Soft-Analytic Plaintext-Checking Oracle Attack on ML-KEM

Presented by:

FEPOU Orlie

Registration: UBA24EP005

Supervised by:

Dr. Julius Hermelink
MPI-SP

Prof. Dr. Peter Schwabe
MPI-SP

Prof. Emmanuel Fouotsa
The University of Bamenda

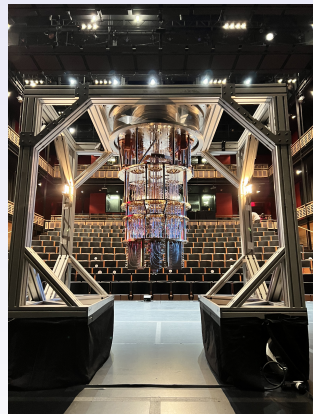
July 21, 2026

Outline

- 1 Problem Statement and Motivation
- 2 Objectives and Research Questions
- 3 Literature Review
- 4 Methodology and Technical Approach
- 5 System Design and Implementation
- 6 Results and Analysis
- 7 Contributions
- 8 Conclusion

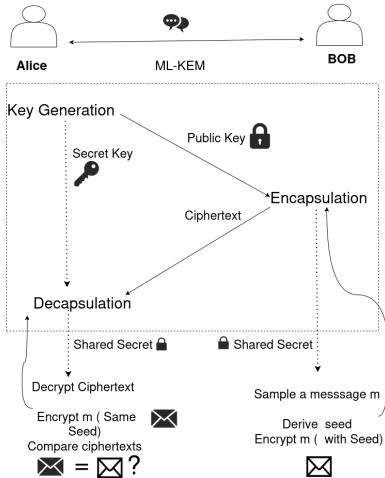
Problem Statement and Motivation

- Large-scale quantum computers threaten classical public key schemes [6]
- NIST mitigation: post-quantum standardisation process (2016) [1]
- ML-KEM: first standardised KEM in 2024 [4]
- ML-KEM is widely deployed in Chrome, iMessage, Signal, in 60% of HTTPS traffic [2]



IBM quantum computer

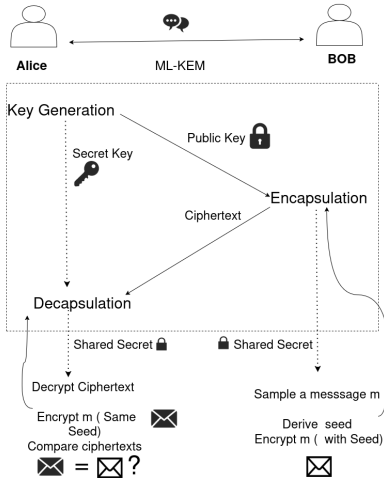
Problem Statement and Motivation



ML-KEM Cryptosystem

- Considered to be secure (IND-CCA2)

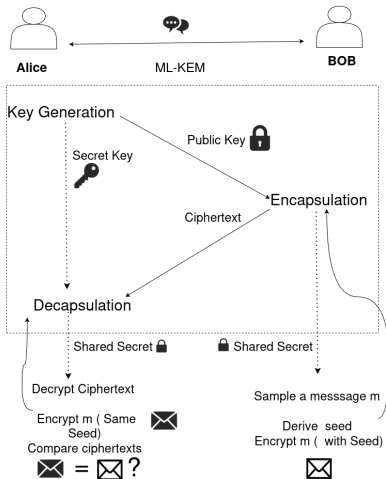
Problem Statement and Motivation



ML-KEM Cryptosystem

- Considered to be secure (IND-CCA2)
- Based on MLWE: operations on polynomials

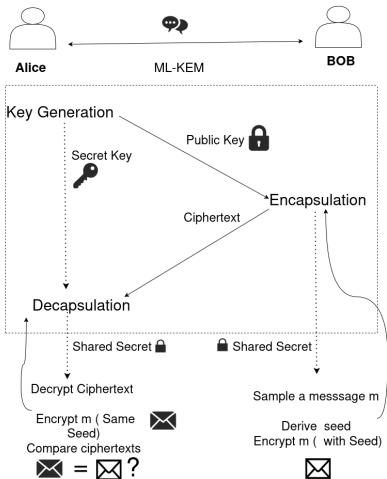
Problem Statement and Motivation



ML-KEM Cryptosystem

- Considered to be secure (IND-CCA2)
- Based on MLWE: operations on polynomials
- Uses the Number Theoretic Transform (NTT): efficient polynomial multiplication

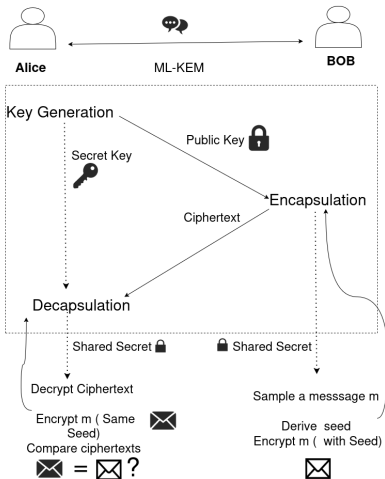
Problem Statement and Motivation



ML-KEM Cryptosystem

- Considered to be secure (IND-CCA2)
- Based on MLWE: operations on polynomials
- Uses the Number Theoretic Transform (NTT): efficient polynomial multiplication
- FO Transform: re-encrypt and compare

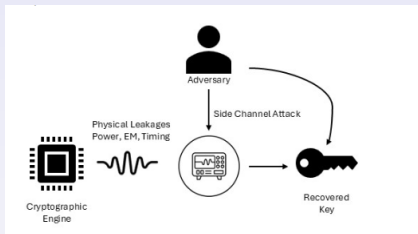
Problem Statement and Motivation



ML-KEM Cryptosystem

- Considered to be secure (IND-CCA2)
- Based on MLWE: operations on polynomials
- Uses the Number Theoretic Transform (NTT): efficient polynomial multiplication
- FO Transform: re-encrypt and compare
- FO results in large side-channel attack surface

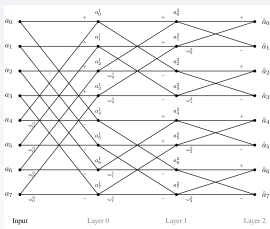
Problem Statement and Motivation



Side-Channel Attacks

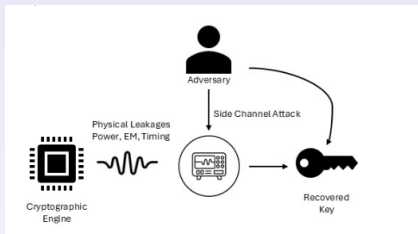
- Recover sensitive information from side channel sources (power, EM, timing)

Side-channel attack model



NTT butterfly network

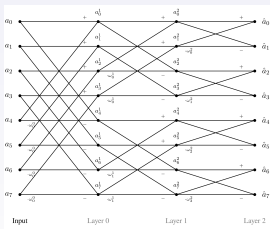
Problem Statement and Motivation



Side-Channel Attacks

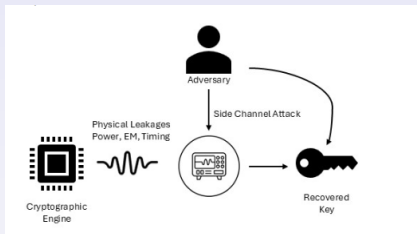
- Recover sensitive information from side channel sources (power, EM, timing)
- NTT operations in ML-KEM leak exploitable side channel information

Side-channel attack model

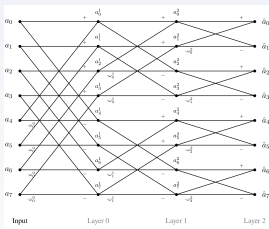


NTT butterfly network

Problem Statement and Motivation



Side-channel attack model

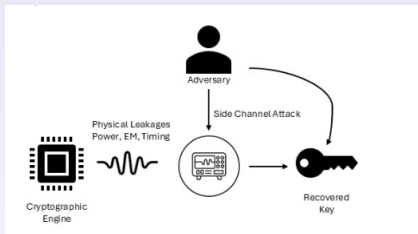


NTT butterfly network

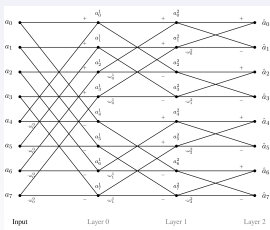
Side-Channel Attacks

- Recover sensitive information from side channel sources (power, EM, timing)
- NTT operations in ML-KEM leak exploitable side channel information
- SASCA: uses a factor graph BP algorithm to propagate probabilities from side-channel leakage + algebraic constraints over targeted variables

Problem Statement and Motivation



Side-channel attack model

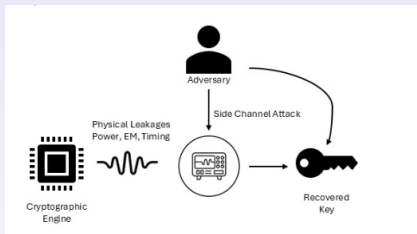


NTT butterfly network

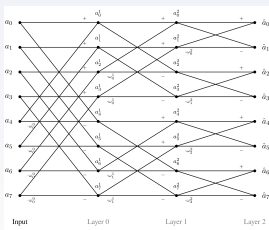
Side-Channel Attacks

- Recover sensitive information from side channel sources (power, EM, timing)
- NTT operations in ML-KEM leak exploitable side channel information
- SASCA: uses a factor graph BP algorithm to propagate probabilities from side-channel leakage + algebraic constraints over targeted variables
- An effective class of Noisy SCA on ML-KEM

Problem Statement and Motivation



Side-channel attack model



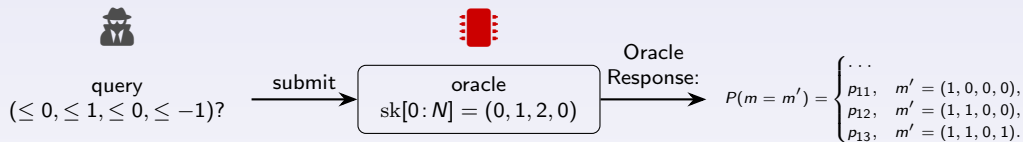
NTT butterfly network

Side-Channel Attacks

- Recover sensitive information from side channel sources (power, EM, timing)
- NTT operations in ML-KEM leak exploitable side channel information
- SASCA: uses a factor graph BP algorithm to propagate probabilities from side-channel leakage + algebraic constraints over targeted variables
- An effective class of Noisy SCA on ML-KEM
- Countermeasures: masking and hiding

Problem Statement and Motivation

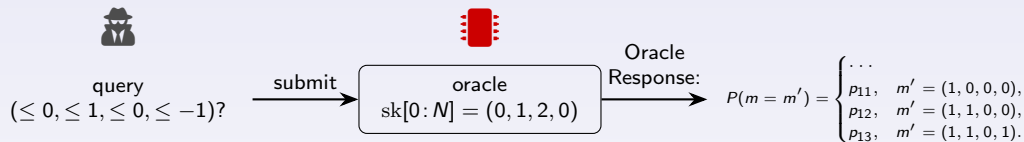
Plaintext-Checking Oracle Attacks: one of the most studied SCA attacks on ML-KEM



- Adversary (Eve): wants Alice's secret key

Problem Statement and Motivation

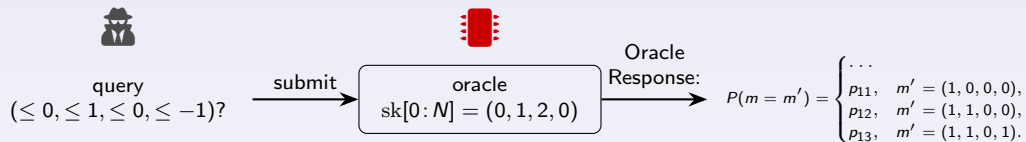
Plaintext-Checking Oracle Attacks: one of the most studied SCA attacks on ML-KEM



- Adversary (Eve): wants Alice's secret key
- Crafted ciphertexts submitted for decapsulation leak information about the secret key

Problem Statement and Motivation

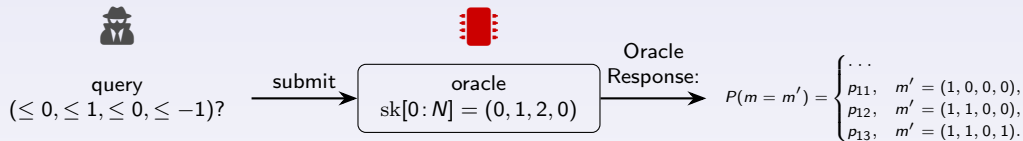
Plaintext-Checking Oracle Attacks: one of the most studied SCA attacks on ML-KEM



- Adversary (Eve): wants Alice's secret key
- Crafted ciphertexts submitted for decapsulation leak information about the secret key
- Instantiates a Noisy SCA oracle to submit queries

Problem Statement and Motivation

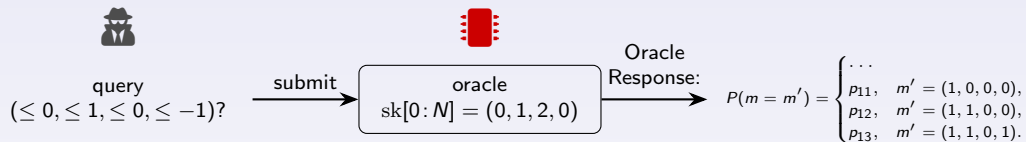
Plaintext-Checking Oracle Attacks: one of the most studied SCA attacks on ML-KEM



- Adversary (Eve): wants Alice's secret key
- Crafted ciphertexts submitted for decapsulation leak information about the secret key
- Instantiates a Noisy SCA oracle to submit queries
- Oracle response: narrows the interval on secret key coefficients

Problem Statement and Motivation

Plaintext-Checking Oracle Attacks: one of the most studied SCA attacks on ML-KEM



- Adversary (Eve): wants Alice's secret key
- Crafted ciphertexts submitted for decapsulation leak information about the secret key
- Instantiates a Noisy SCA oracle to submit queries
- Oracle response: narrows the interval on secret key coefficients
- Adaptive queries $\Rightarrow$ full key recovery

Main: To what extent can soft-analytic techniques enhance noise tolerance and query efficiency of PC-oracle attacks on ML-KEM?

Specific:

- ① How to exploit NTT algebraic relations for soft-analytic PC-oracle attacks?

Main: To what extent can soft-analytic techniques enhance noise tolerance and query efficiency of PC-oracle attacks on ML-KEM?

Specific:

- ① How to exploit NTT algebraic relations for soft-analytic PC-oracle attacks?
- ② How do different approaches compare in query complexity and error rates under varying noise?

Main: To what extent can soft-analytic techniques enhance noise tolerance and query efficiency of PC-oracle attacks on ML-KEM?

Specific:

- ① How to exploit NTT algebraic relations for soft-analytic PC-oracle attacks?
- ② How do different approaches compare in query complexity and error rates under varying noise?
- ③ To what extent do masking countermeasures protect against soft-analytic PC-oracle attacks?

Objectives

Overall: Develop and evaluate soft-analytic PC-oracle attacks exploiting full probability distributions from side-channel measurements.

Specific:

- ❶ Instantiate realistic oracles from noisy HW leakage across the full NTT network

Objectives

Overall: Develop and evaluate soft-analytic PC-oracle attacks exploiting full probability distributions from side-channel measurements.

Specific:

- ❶ Instantiate realistic oracles from noisy HW leakage across the full NTT network
- ❷ Extend distribution oracles to exploit NTT algebraic relations via a unified factor graph

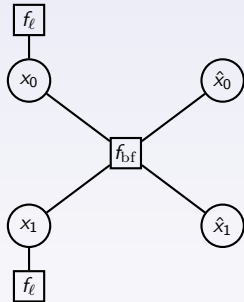
Objectives

Overall: Develop and evaluate soft-analytic PC-oracle attacks exploiting full probability distributions from side-channel measurements.

Specific:

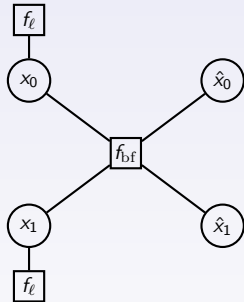
- ① Instantiate realistic oracles from noisy HW leakage across the full NTT network
- ② Extend distribution oracles to exploit NTT algebraic relations via a unified factor graph
- ③ Assess masking countermeasures against the proposed attack

NTT-SASCA factor graph of Primas et al.(2019)[5]:



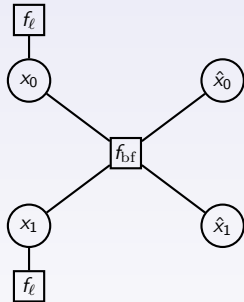
- Efficient factor graph and BP techniques

NTT-SASCA factor graph of Primas et al.(2019)[5]:



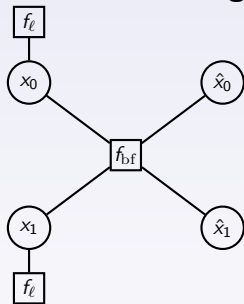
- Efficient factor graph and BP techniques
- Leakage factors f_ℓ : priors from template matching

NTT-SASCA factor graph of Primas et al.(2019)[5]:



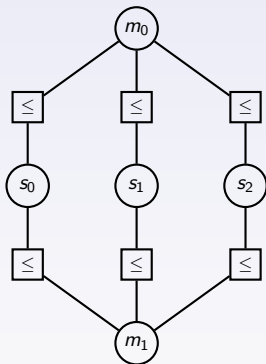
- Efficient factor graph and BP techniques
- Leakage factors f_ℓ : priors from template matching
- 7 layers, 128 butterflies per layer

NTT-SASCA factor graph of Primas et al.(2019)[5]:



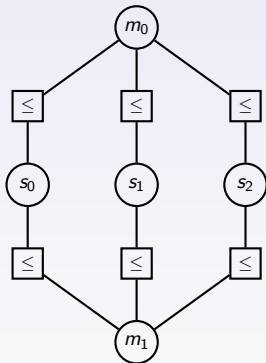
- Efficient factor graph and BP techniques
- Leakage factors f_ℓ : priors from template matching
- 7 layers, 128 butterflies per layer
- 100% recovery up to $\sigma \leq 1.5$ (unmasked) and $\sigma \leq 0.4$ (second-order masking)

Soft analytic PC-oracle key-recovery graph of Hermelink et al(2025) [3]:



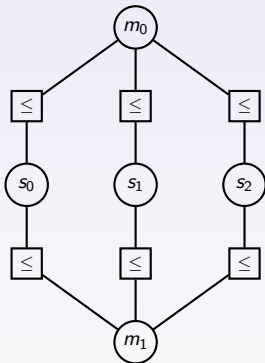
- Introduced distribution oracle, soft-analytic techniques, focus on noise tolerance

Soft analytic PC-oracle key-recovery graph of Hermelink et al(2025) [3]:



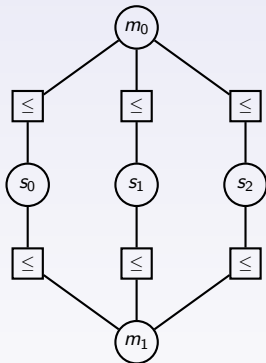
- Introduced distribution oracle, soft-analytic techniques, focus on noise tolerance
- BP: factor graph made of secret s_i , inequality factors $\leq$, message nodes m_j

Soft analytic PC-oracle key-recovery graph of Hermelink et al(2025) [3]:



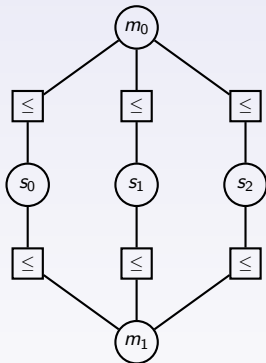
- Introduced distribution oracle, soft-analytic techniques, focus on noise tolerance
- BP: factor graph made of secret s_i , inequality factors $\leq$, message nodes m_j
- Oracle from last-layer NTT leakage only

Soft analytic PC-oracle key-recovery graph of Hermelink et al(2025) [3]:



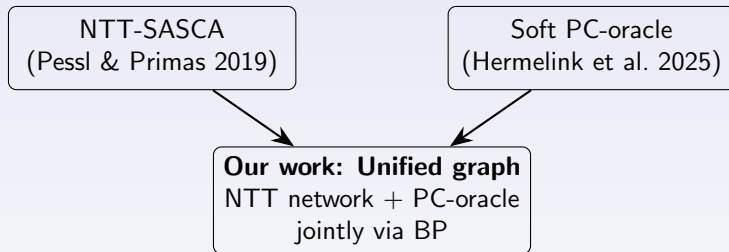
- Introduced distribution oracle, soft-analytic techniques, focus on noise tolerance
- BP: factor graph made of secret s_i , inequality factors $\leq$, message nodes m_j
- Oracle from last-layer NTT leakage only
- Soft-analytic oracles, noise-tolerant (up to $\sigma = 10$)

Soft analytic PC-oracle key-recovery graph of Hermelink et al(2025) [3]:



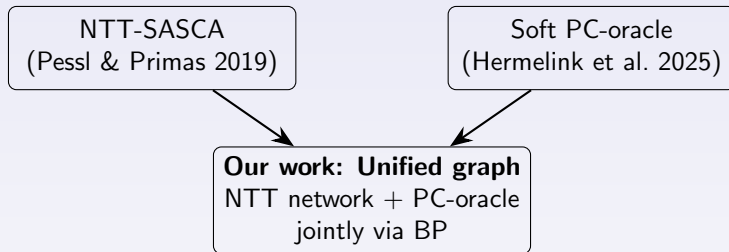
- Introduced distribution oracle, soft-analytic techniques, focus on noise tolerance
- BP: factor graph made of secret s_i , inequality factors $\leq$, message nodes m_j
- Oracle from last-layer NTT leakage only
- Soft-analytic oracles, noise-tolerant (up to $\sigma = 10$)
- **Limitation:** does not exploit the NTT algebraic structure

Literature Review and Research Gap



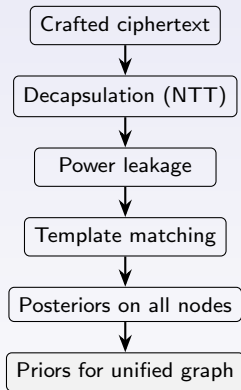
- Prior work: NTT leakage extraction and key recovery are two independent, sequential phases

Literature Review and Research Gap



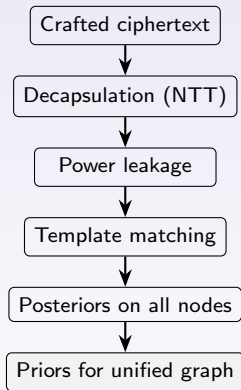
- Prior work: NTT leakage extraction and key recovery are two independent, sequential phases
- Our work: a single factor graph where information flows bidirectionally between the NTT and the key-recovery graphs

Methodology: Oracle Instantiation



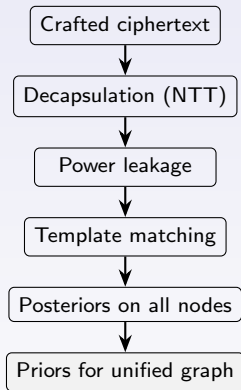
- Noisy HW leakage: $\ell = \text{HW}(x) + \mathcal{N}(0, \sigma^2)$

Methodology: Oracle Instantiation



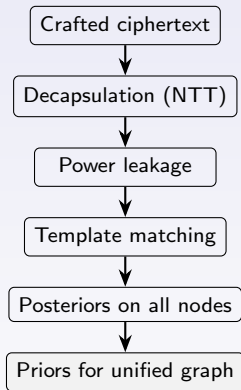
- Noisy HW leakage: $\ell = \text{HW}(x) + \mathcal{N}(0, \sigma^2)$
- Template matching over intermediate values

Methodology: Oracle Instantiation



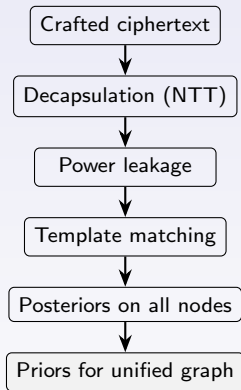
- Noisy HW leakage: $\ell = \text{HW}(x) + \mathcal{N}(0, \sigma^2)$
- Template matching over intermediate values
- Prior work: last layer only

Methodology: Oracle Instantiation



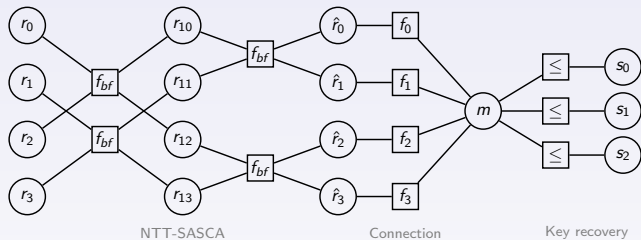
- Noisy HW leakage: $\ell = \text{HW}(x) + \mathcal{N}(0, \sigma^2)$
- Template matching over intermediate values
- Prior work: last layer only
- we use the full distributions on every NTT node

Methodology: Oracle Instantiation



- Noisy HW leakage: $\ell = \text{HW}(x) + \mathcal{N}(0, \sigma^2)$
- Template matching over intermediate values
- Prior work: last layer only
- we use the full distributions on every NTT node
- To build priors on the unified graph

Methodology: Unified Factor Graph



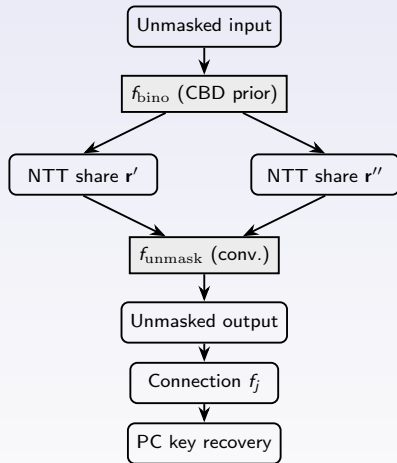
Connection factor:

$$f_i(\hat{r}_i, m) = \begin{cases} 1 & \hat{r}_i = [\text{NTT}(\text{CBD}(G(m, h)))] \\ 0 & \text{otherwise} \end{cases}$$

BP messages:

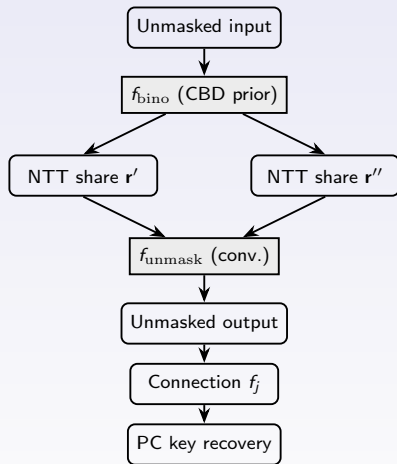
- NTT $\rightarrow$ msg: $\mu_{f_i \rightarrow m}(m') = p(\hat{r}_i = c_i(m'))$
- Product: $p(m') \propto \prod_i \mu_{f_i \rightarrow m}(m')$
- Msg $\rightarrow$ NTT: $\mu_{f_i \rightarrow \hat{r}_i}(v) = \sum_{m'} p(m')$

Methodology: Masking Countermeasures



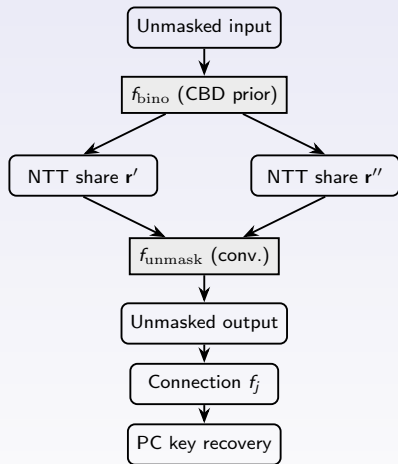
- $\mathbf{r} = \mathbf{r}' + \mathbf{r}'' \bmod q$

Methodology: Masking Countermeasures



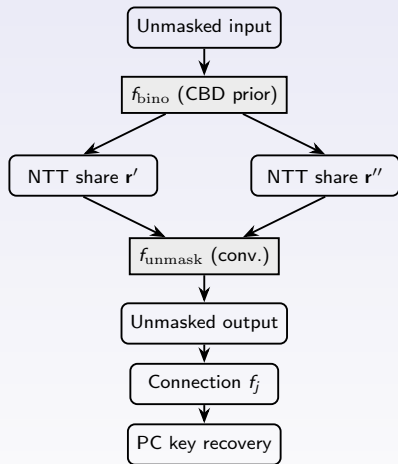
- $\mathbf{r} = \mathbf{r}' + \mathbf{r}'' \bmod q$
- Each share: own NTT sub-graph

Methodology: Masking Countermeasures



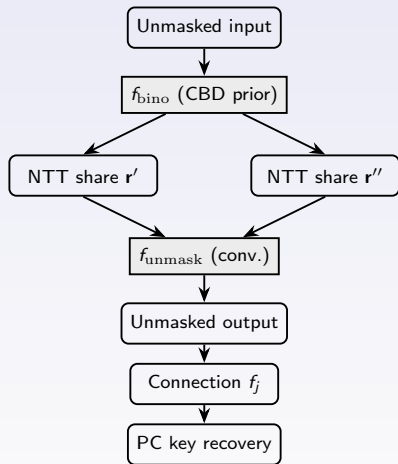
- $\mathbf{r} = \mathbf{r}' + \mathbf{r}'' \bmod q$
- Each share: own NTT sub-graph
- f_{bino} : joins shares at input

Methodology: Masking Countermeasures



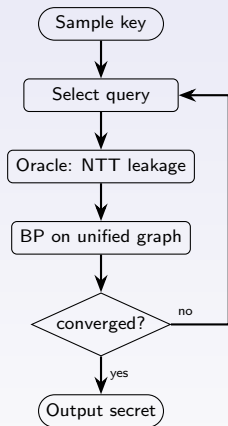
- $\mathbf{r} = \mathbf{r}' + \mathbf{r}'' \bmod q$
- Each share: own NTT sub-graph
- f_{bino} : joins shares at input
- f_{unmask} : joins at output

Methodology: Masking Countermeasures



- $\mathbf{r} = \mathbf{r}' + \mathbf{r}'' \bmod q$
- Each share: own NTT sub-graph
- f_{bino} : joins shares at input
- f_{unmask} : joins at output
- Higher orders: more sub-graphs

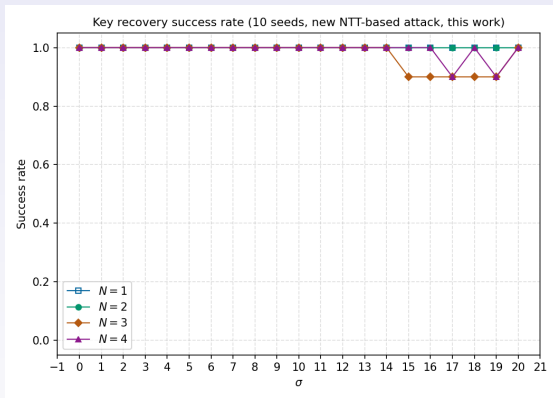
System Design and Implementation



- Python: oracle, ciphertext construction, query selection, recovery loop
- Rust (PyO3 + Rayon): belief propagation implementation, connection factor, key-recovery BP
- Graph rebuilt after each query
- PQClean for all crypto (unmodified)
- $L = 1, 20$ BP iterations, 40-call budget

Hardware: ThinkPad P14s (Ryzen 7, 28 GiB); evaluation on a shared server using 400 cores.

Results and Analysis

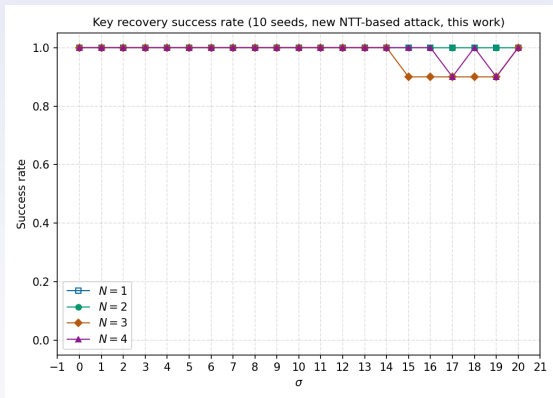


Success rate across noise levels and N values

Success rate: 835/840 successfully key recovery

- $N = 1, N = 2$: 100% at all noise levels

Results and Analysis

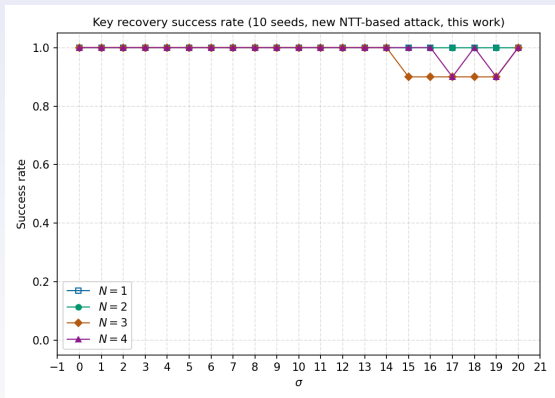


Success rate across noise levels and N values

Success rate: 835/840 successfully key recovery

- $N = 1, N = 2$: 100% at all noise levels
- $N = 4$: 100% except $\sigma = 19$ (90%)

Results and Analysis

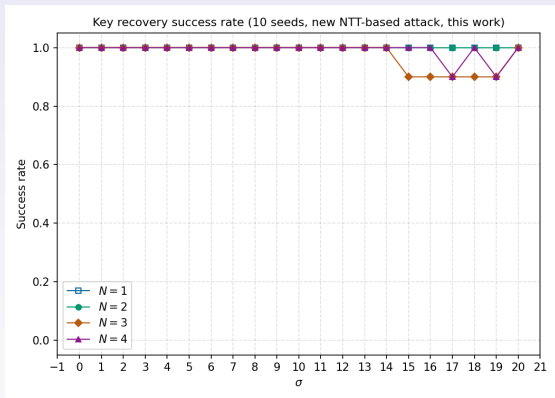


Success rate across noise levels and N values

Success rate: 835/840 successfully key recovery

- $N = 1, N = 2$: 100% at all noise levels
- $N = 4$: 100% except $\sigma = 19$ (90%)
- $N = 3$: 100% except $\sigma \in \{16-19\}$ (90%)

Results and Analysis

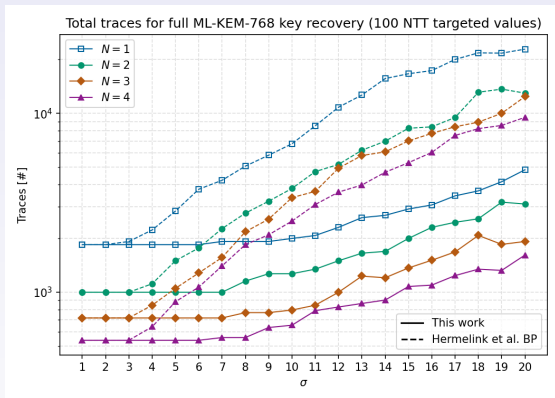


Success rate across noise levels and N values

Success rate: 835/840 successfully key recovery

- $N = 1, N = 2$: 100% at all noise levels
- $N = 4$: 100% except $\sigma = 19$ (90%)
- $N = 3$: 100% except $\sigma \in \{16-19\}$ (90%)
- All 5 failures: false convergence

Results and Analysis



Total traces: unified vs Hermelink et al.

Trace reduction:

N	σ	Ours	Prior	Red.
4	5	538	883	1.6×
4	10	653	2 496	3.8×
4	15	1 075	5 299	4.9×
4	20	1 613	9 504	5.9×
3	20	1 920	12 416	6.5×

Results and Analysis: Masking Assessment

- First-order masking: two NTT sub-graphs per query
- Preliminary trials: very low success rate
- False convergence amplified by masking
- Higher-order masking: further increases instability
- Other countermeasures not explored
- **Prerequisite:** stabilise unmasked graph first

Contributions of the Work

- Tested noise tolerance: up to $\sigma = 20$ and success rate $\geq 90\%$

Contributions of the Work

- Tested noise tolerance: up to $\sigma = 20$ and success rate $\geq 90\%$
- **Unified factor graph** connecting NTT and soft PC-oracle key recovery through a connection factor

Contributions of the Work

- Tested noise tolerance: up to $\sigma = 20$ and success rate $\geq 90\%$
- **Unified factor graph** connecting NTT and soft PC-oracle key recovery through a connection factor
- **Up to $6.5\times$ fewer traces** than the Key Recovery graph alone [3]

Contributions of the Work

- Tested noise tolerance: up to $\sigma = 20$ and success rate $\geq 90\%$
- **Unified factor graph** connecting NTT and soft PC-oracle key recovery through a connection factor
- **Up to $6.5\times$ fewer traces** than the Key Recovery graph alone [3]
- **Open-source implementation** in Python + Rust

Contributions of the Work

- Tested noise tolerance: up to $\sigma = 20$ and success rate $\geq 90\%$
- **Unified factor graph** connecting NTT and soft PC-oracle key recovery through a connection factor
- **Up to $6.5\times$ fewer traces** than the Key Recovery graph alone [3]
- **Open-source implementation** in Python + Rust
- **Masking architecture designed**: false convergence identified as main obstacle

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)

Future work:

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable

Future work:

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable
- Unmasked ML-KEM-768 only; $N \leq 4$

Future work:

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable
- Unmasked ML-KEM-768 only; $N \leq 4$
- Numerical underflow prevents $N \geq 5$

Future work:

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable
- Unmasked ML-KEM-768 only; $N \leq 4$
- Numerical underflow prevents $N \geq 5$

Future work:

- Investigate and mitigate false convergence

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable
- Unmasked ML-KEM-768 only; $N \leq 4$
- Numerical underflow prevents $N \geq 5$

Future work:

- Investigate and mitigate false convergence
- Log-domain multiplication for $N \geq 5$

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable
- Unmasked ML-KEM-768 only; $N \leq 4$
- Numerical underflow prevents $N \geq 5$

Future work:

- Investigate and mitigate false convergence
- Log-domain multiplication for $N \geq 5$
- Evaluate masked graph once stability is achieved

Limitations and Future Work

Limitations:

- Simulation only (noisy HW model, not a physical device)
- Single NTT layer ($L = 1$); deeper propagation unstable
- Unmasked ML-KEM-768 only; $N \leq 4$
- Numerical underflow prevents $N \geq 5$

Future work:

- Investigate and mitigate false convergence
- Log-domain multiplication for $N \geq 5$
- Evaluate masked graph once stability is achieved
- Wider evaluation: 100 samples per setting

- **Problem:** the soft-analytic PC-oracle, though powerful, exploits only the last NTT layer and discards algebraic information
- **Approach:** a unified factor graph where BP is applied over the NTT butterfly network and key recovery jointly
- **Results:** 835/840 successful recoveries; up to $6.5\times$ fewer traces than the Key recovery graph alone
- **Open problem:** false convergence in the loopy joint graph and countermeasure assessment to mitigate the proposed attack

Thank you for your keen attention!
Questions?

References I

- [1] Lily Chen et al. *Report on Post-Quantum Cryptography*. Tech. rep. NISTIR 8105. National Institute of Standards and Technology, 2016. DOI: 10.6028/NIST.IR.8105.
- [2] Cloudflare. *Bringing More Transparency to Post-Quantum Usage, Encrypted Messaging, and Routing Security*. Accessed June 2026. 2026. URL: <https://blog.cloudflare.com/radar-origin-pq-key-transparency-aspa/>.
- [3] Julius Hermelink, Erik Mårtensson, and Maggie Tran. *Noise-Tolerant Plaintext-Checking Oracle Attacks – A Soft-Analytic Approach Applied to ML-KEM*. Cryptology ePrint Archive, Paper 2025/1496. 2025. URL: <https://eprint.iacr.org/2025/1496>.
- [4] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Federal Information Processing Standards Publication FIPS 203. National Institute of Standards and Technology, 2024. DOI: 10.6028/NIST.FIPS.203.
- [5] Peter Pessl and Robert Primas. “More practical single-trace attacks on the number theoretic transform”. In: *International Conference on Cryptology and Information Security in Latin America*. Springer. 2019, pp. 130–149.

- [6] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.